

MediLink

Documentacion del proyecto

Version de cierre para hackathon · 9 de junio de 2026

MediLink es una demo de interoperabilidad y humanizacion medica construida con backend FastAPI y frontend React. El sistema ayuda al profesional a transformar un informe tecnico en una explicacion comprensible para el paciente, exportar un recurso FHIR DiagnosticReport y conectar el resultado con Idonia cuando el tenant externo lo permite.

1. Problema que resuelve

En la practica clinica, gran parte de la informacion diagnostica llega al paciente en un formato demasiado tecnico. Eso genera ansiedad, incomension y mas carga para el medico, que debe reinterpretar el contenido en consulta o por otros canales. Al mismo tiempo, la interoperabilidad entre sistemas sigue siendo una barrera real.

2. Solucion propuesta

MediLink convierte un flujo clinico fragmentado en una experiencia mas clara y conectada:

- El medico consulta pacientes y procesa informes desde una unica interfaz.
- El backend genera una version humanizada del informe para paciente.
- El sistema produce tambien un FHIR DiagnosticReport exportable.
- El paciente ve el informe en lenguaje claro, sus recetas explicadas y puede interactuar con Avatar Elena.
- Cuando Idonia staging esta habilitado, el sistema prepara la subida del bundle clinico y el acceso mediante Magic Link.

3. Arquitectura

Capa	Tecnologia	Funcion
Frontend	React 18 + TypeScript + Vite	Dashboard medico, dashboard paciente, errores trazables, experiencia de demo

Backend	FastAPI + Pydantic	Auth, pacientes, informes, FHIR, Avatar, recetas, integraciones
IA	Azure OpenAI + Recog con fallback	Humanizacion del informe, ayuda conversacional, resiliencia en demo
Interoperabilidad	FHIR R4 + Idonia staging	Export estructurado y preparacion de acceso remoto al contenido

4. Modulos principales

Autenticacion y roles

Se implemento JWT con control de acceso por rol. El medico solo puede ver sus pacientes asignados y el paciente solo puede acceder a su propio registro.

Busqueda clinica

Se anadio una busqueda paginada por nombre o DNI, tolerante a mayusculas, minusculas y acentos, para acelerar el flujo del medico en demo.

Procesado de informes

El sistema genera tres salidas sobre el mismo caso clinico: informe humanizado, PDF para paciente y FHIR DiagnosticReport.

Avatar Elena

El paciente puede consultar dudas en una experiencia guiada y contextualizada, con guardrails para evitar respuestas inadecuadas o fuera del alcance definido.

Recetas humanizadas

Las recetas conservan la pauta clinica original pero se presentan con un lenguaje mas accesible.

Idonia

El backend queda preparado para subir un bundle formado por informe tecnico, informe para paciente y estudio radiologico, y despues solicitar un Magic Link de acceso.

5. Evolucion del proyecto por fases

- **Fase 1:** base funcional y busqueda clinica.
- **Fase 2:** refuerzo de seguridad con RBAC y JWT.
- **Fase 3:** estandarizacion de errores y trazabilidad con trace_id.
- **Fase 4:** mejora de OpenAPI y ejemplos de contrato.
- **Fase 5:** mejora de UX medico y paciente para demo.
- **Fase II y III Idonia:** humanizacion auditable local, bundle preparado y bloqueo externo aislado.

6. Que se ha validado

- Login y roles funcionando.
- Busqueda clinica funcional.
- Procesado de informe con salida humanizada y FHIR.
- PDF de informe para paciente.
- Avatar Elena operativo.
- Errores trazables con trace_id visibles tambien en frontend.
- Artefacto humanizado local generado aunque el tercero falle.
- Frontend compilado correctamente y smoke tests backend superados.

7. Estado actual de la integracion con Idonia

La direccion de trabajo ha sido correcta: primero asegurar la logica interna del sistema y despues validar el tramo remoto. A fecha de cierre, el backend ya construye el bundle y solicita el acceso, pero el tenant de staging sigue bloqueando la parte remota. La evidencia apunta a un problema externo de autorizacion o permisos, no a una carencia clara del flujo local.

Esto significa que la demo local completa esta lista, mientras que la demo extremo a extremo con Idonia queda pendiente de desbloqueo del entorno externo.

8. Ficheros principales tocados

- backend/main.py
- backend/models/schemas.py
- backend/routers/patients.py
- backend/routers/doctor.py
- backend/routers/reports.py
- backend/routers/idonia_v1.py

- backend/services/authz_service.py
- backend/services/recog_service.py
- backend/services/idonia_service.py
- frontend/src/services/api.ts
- frontend/src/pages/DoctorDashboard.tsx
- frontend/src/pages/PatientDashboard.tsx
- frontend/src/pages/LoginPage.tsx
- frontend/src/App.tsx
- frontend/src/types.ts

9. Seguridad y buenas practicas

- Control de acceso por rol.
- Contrato uniforme de error con trace_id para soporte.
- Credenciales solo en entorno local o variables de despliegue.
- Documentacion saneada para no exponer secretos.
- Datos de demo ficticios y alcance acotado para uso responsable.

10. Preparacion para despliegue

El proyecto queda preparado para desplegar backend en Azure App Service y frontend en Azure Static Web Apps o App Service. La integracion con Idonia puede revalidarse en cuanto el tenant de staging habilite acceso correcto.

11. Conclusiones

MediLink ya demuestra valor real en tres planos: comprension para el paciente, eficiencia para el medico e interoperabilidad para el ecosistema. El producto esta listo para una demo funcional y defendible, con una unica dependencia externa pendiente para cerrar el recorrido completo con Idonia.